

Datatypes in Allegories

Bird & de Moor, *Algebra of Programming* ch. 5 —
in Freyd diagram order

Conventions

Composition is Juxtaposition in diagram order: $RS =$ “first R , then S ”. Objects α, β, γ ; relations Q, R, S, T, U, V, X, Y ; **maps** (entire and simple) f, g, h, k, m ; coreflexives $C \subset 1$. R° converse, $\subset$ containment, $\cap, \cup$ meet/join, $-$ subtraction, $\emptyset$ empty, Π largest, 1 identity, dom, ran . $[\alpha]$ power object, $\ni : [\alpha] \rightarrow \alpha$ membership, Λ the power transpose ($f = \Lambda R \iff f \ni = R$), $\tau = \Lambda 1$ singleton, $\bigcup = E \ni$ union, $ER = \Lambda(\ni R)$ existential image. F, G relators; $\text{in} : F(\mu F) \rightarrow \mu F$ the initial algebra, $(\llbracket R \rrbracket)$ its catamorphism; $(\mu X : \varphi X)$ the least solution of $X = \varphi X$. Points: $x R y$.

B&dM dictionary. Everything else on the sheet is the book’s, mirrored through this table and nothing more.

B&dM	here
$S \cdot R$	RS composition reverses
id	1
outl, outr	π, π'
inl, inr	ι, ι'
α	in the initial algebra, not an object
$(\llbracket R \rrbracket)$	$(\llbracket R \rrbracket)$
μ	$\bigcup$ (μ is kept for the least fixed point)
$\in, \ni$	$\ni, \ni^\circ$
$T \setminus U, U/T$	$U/T, T \setminus U$ slashes flip, roles kept
$(a, b) \langle R, S \rangle c$	$c \langle R, S \rangle (a, b)$ points reverse too

A row tagged $\checkmark$ has its mirrored statement machine-checked in `AOP/A*.lean`; the rest are hand-mirrored from the page.

5.1 Relators

5.1 ✓ **RELATOR** $F : \mathcal{A} \rightarrow \mathcal{B}$ between tabular allegories
 $\triangleq$ a *monotonic* functor: $R \subset S \implies FR \subset FS$.

Lem 5.1 ✓ F a relator and f a map $\implies Ff$ is a map and
 $(Ff)^\circ = F(f^\circ)$.

Thm 5.1 ✓ A functor is a relator $\iff$ it preserves converse:
 $F(R^\circ) = (FR)^\circ$.

Cor 5.1 ✓ Relators agreeing on maps are equal: $(\forall f. Ff = Gf) \implies F = G$. Tabulate: $FR = F(f^\circ g) = (Ff)^\circ (Fg)$.

Hence FR° is unambiguous — it names $(FR)^\circ$ and $F(R^\circ)$ alike.

Ex 5.2 ✓ A relator preserves meets of coreflexives, $F(C \cap D) = FC \cap FD$. The restriction $C, D \subset 1$ is needed.

Ex 5.5 ✓ $F(\text{dom } R) = \text{dom}(FR)$.

Ex 5.4 Not every functor on **Fun** extends to a relator: for $FA = \emptyset$ if $A = \emptyset$ and $\{0\}$ otherwise, the constant maps $\text{one}, \text{two} : \{0\} \rightarrow \{1, 2\}$ give $F(\text{two one}^\circ) = 1$, while preservation of converse forces $\emptyset$.

5.2 Relational products

(π, π') tabulates Π , so $\langle f, g \rangle = (f\pi^\circ) \cap (g\pi'^\circ)$ for maps; (5.1) reads that same formula for arbitrary relations.

$$(5.1) \quad \checkmark \quad \langle R, S \rangle \triangleq (R\pi^\circ) \cap (S\pi'^\circ). \quad \text{Rel } c \langle R, S \rangle (a, b) \iff cRa \wedge cSb.$$

$$(5.2) \quad \checkmark \quad R \times S \triangleq \langle \pi R, \pi' S \rangle.$$

$\times$ is monotonic in both arguments and $(R \times S)^\circ = R^\circ \times S^\circ$, so it is a relator — but it is *not* a categorical product.

$$(5.3) \quad \checkmark \quad \text{ABSORPTION} \quad \langle X, Y \rangle (R \times S) = \langle XR, YS \rangle; \text{ hence } (R \times S)(U \times V) = (RU) \times (SV).$$

$$(5.4) \quad \checkmark \quad \langle XR, Y \rangle \subset \langle X, Y \rangle (R \times 1) \text{ — one of the two special cases (5.3) is staged through.}$$

$$(5.5) \quad \checkmark \quad \langle X, YS \rangle \subset \langle X, Y \rangle (1 \times S), \text{ symmetric to (5.4).}$$

$$(5.6) \quad \checkmark \quad \langle R, S \rangle \pi = (\text{dom } S)R.$$

$$(5.7) \quad \checkmark \quad \langle R, S \rangle \pi' = (\text{dom } R)S.$$

So $\langle R, \emptyset \rangle \pi = \emptyset$, not R : precisely why (π, π') is no categorical product in an allegory.

$$(5.8) \quad \checkmark \quad \text{CANCELLATION} \quad \langle X, Y \rangle \langle R, S \rangle^\circ = (XR^\circ) \cap (YS^\circ).$$

$$\text{Ex 5.6} \quad \checkmark \quad (1 \times S)(R \times 1) \supset R \times S, \text{ from (5.4) and (5.2) alone.}$$

$$\text{Ex 5.7} \quad \langle R, S \rangle^\circ \langle P, Q \rangle \subset (R^\circ P) \times (S^\circ Q).$$

$$\text{Ex 5.8} \quad \langle X, Y \rangle (R \times S) \subset \langle XR, YS \rangle \text{ — the easy half of (5.3).}$$

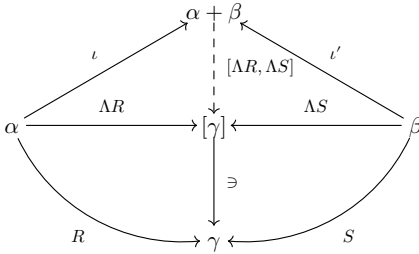
$$\text{Ex 5.9} \quad \checkmark \quad f \langle R, S \rangle = \langle fR, fS \rangle \text{ for a map } f; \text{ false for an arbitrary arrow.}$$

$$\text{Ex 5.10} \quad \text{unzip}(F) \triangleq \langle F\pi, F\pi' \rangle, \text{ and } F(R \times S) \text{ unzip}(F) = \text{unzip}(F)(FR \times FS).$$

5.3 Relational coproducts

A coproduct of α, β in $\text{Fun}(\mathcal{A})$ is a coproduct in the whole power allegory $\mathcal{A}$:

$$\iota T = R \wedge \iota' T = S \iff T = [\Lambda R, \Lambda S] \ni.$$



$$5.3 \checkmark \text{ JUNC } [R, S] \triangleq [\Lambda R, \Lambda S] \ni.$$

$$(5.9) \checkmark [R, S] = (\iota^\circ R) \cup (\iota'^\circ S).$$

$$(5.10) \checkmark R + S \triangleq [R\iota, S\iota'], \text{ monotonic in both arguments, so } + \text{ is a relator.}$$

$$(5.11) \checkmark \text{ CANCELLATION } [U, V]^\circ [R, S] = (U^\circ R) \cup (V^\circ S). \\ \text{Easier than (5.8) — but still not the dual of it, an allegory being its own opposite.}$$

$$\text{Ex 5.12 } \checkmark \iota[1, \emptyset] = 1 \text{ and } \iota'[1, \emptyset] = \emptyset, \text{ and } [1, \emptyset]\iota \cup [\emptyset, 1]\iota' = [\iota, \iota'] = 1; \text{ hence } [1, \emptyset] = \iota^\circ, [\emptyset, 1] = \iota'^\circ, \text{ which gives (5.9).}$$

$$\text{Ex 5.14 } (R + S) \cap ([P, Q][U, V]^\circ) = (R \cap PU^\circ) + (S \cap QV^\circ).$$

5.4 The power relator

- 5.4 ✓ On maps $Pf = ((\exists f)/\exists) \cap (\exists^\circ \setminus (f\exists^\circ))$; read as a definition for relations,

$$PR \triangleq ((\exists R)/\exists) \cap (\exists^\circ \setminus (R\exists^\circ)).$$

Rel $x (PR) y \iff (\forall a \in x. \exists b \in y. aRb) \wedge (\forall b \in y. \exists a \in x. aRb)$. Every element of x has an R -partner in y , and conversely.

- ✓ P is monotone in R , straight from monotonicity of division.
- ✓ $P1 = 1$ — this is the antisymmetry of $\text{subset} = \exists/\exists$ restated, so it needs the allegory unitary and tabular.
- ✓ $(PR)(PS) = P(RS)$, so P is a relator. $\subset$ is division cancellation; $\supset$ needs a tabulation (x, z) of $P(RS)$ and the mediating map $y = \Lambda((x\exists R^\circ) \cap (z\exists S))$.
- ✓ P and E agree on maps: $Pf = \Lambda(\exists f) = Ef$.

Ex 5.15 They differ on relations, and Corollary 5.1 does not object: E is not monotonic — distinct maps are never comparable, so $R \subset S$ says nothing about $\Lambda(\exists R)$ and $\Lambda(\exists S)$ — hence not a relator.

Ex 5.16 $P(\text{dom } R)(ER) \subset PR$.

5.5 Relational catamorphisms

Let F have initial algebra $\text{in} : FT \rightarrow T$ in the subcategory of maps. It is then initial for relational algebras as well.

(5.12) ✓ $\text{in } X = (FX)R \iff X = \llbracket \Lambda((F\exists)R) \rrbracket \exists.$

5.5 ✓ $\llbracket R \rrbracket \triangleq \llbracket \Lambda((F\exists)R) \rrbracket \exists$ — the unique X with $\text{in } X = (FX)R$.

5.5 ✓ **EILENBERG–WRIGHT** $\Lambda \llbracket R \rrbracket = \llbracket \Lambda((F\exists)R) \rrbracket$: the transpose of a relational catamorphism is a catamorphism of maps. This is what turns a relational specification into a functional fold over sets, and it is used throughout §5.6.

✓ Over a map algebra the relational catamorphism is the ordinary one.

5.5 **TYPE RELATOR** F binary with initial type (α, T) : then $(TR)^\circ = T(R^\circ)$, so T is a relator. By uniqueness — both sides solve $\text{in } X = F(R^\circ, X)$ in.

Ex 5.17 ✓ In a Boolean allegory each coreflexive C has a $\sim C$ with $C \cap \sim C = \emptyset$ and $C \cup \sim C = 1$. Then $\text{guard } C \triangleq [C, \sim C]^\circ$ is a map, $(C \rightarrow R, S) \triangleq (\text{guard } C)(R + S)$, and conditionals behave like cases:

$$R \subset (C \rightarrow S, T) \iff CR \subset S \wedge (\sim C)R \subset T.$$

Ex 5.18 F preserves meets $\implies$ so does the type relator T it induces; in particular list does.

Ex 5.19 R entire $\implies \llbracket R \rrbracket$ entire (reflection gives $\text{dom } \llbracket R \rrbracket = 1$).

$$\begin{array}{ccc}
 FT & \xrightarrow{\text{in}} & T \\
 \downarrow F\llbracket \Lambda((F\exists)R) \rrbracket & & \downarrow \llbracket \Lambda((F\exists)R) \rrbracket \\
 F[\alpha] & \xrightarrow{\Lambda((F\exists)R)} & [\alpha] \\
 \downarrow F\exists & & \downarrow \exists \\
 F\alpha & \xrightarrow{R} & \alpha
 \end{array}$$

5.6 Combinatorial functions

Lists mean cons-lists, and list is their type functor, $\text{list } R = (\text{nil}, (R \times 1) \text{ cons})$. Each relation below is specified as a catamorphism, transposed by Eilenberg–Wright, then implemented on lists.

Subsequences

5.6 $\text{subseq} = (\llbracket \text{nil}, \text{cons} \cup \pi' \rrbracket) : \text{list } \alpha \rightarrow \text{list } \alpha$ — at each step keep the head or drop it.

$\Lambda \text{ subseq} = (\llbracket \text{nil } \tau, \langle \Lambda(1 \times \exists)(P \text{ cons}), \pi' \rangle \text{ cup} \rrbracket)$, by expanding $\Lambda(\llbracket \text{nil}, \text{cons} \cup \pi' \rrbracket F(1, \exists))$ for $F(A, B) = 1 + (A \times B)$. Pointwise: $e = \{\llbracket \rrbracket\}$ and $f(a, xs) = \{\text{cons}(a, x) \mid x \in xs\} \cup xs$.

$\Lambda(R \cup S) = \langle \Lambda R, \Lambda S \rangle \text{ cup}$, where cup unions a pair of sets.

5.6 Sets replaced by lists: $\text{subseqs} = (\llbracket \text{nil wrap}, \langle \text{cpr}(\text{list cons}), \pi' \rangle \text{ cat} \rrbracket)$, i.e. $e = [\llbracket \rrbracket]$ and $f(a, xs) = [\text{cons}(a, x) \mid x \leftarrow xs] \# xs$.

$\text{cpr} : \alpha \times \text{list } \beta \rightarrow \text{list}(\alpha \times \beta)$ (“cartesian product, right”), $\text{cpr}(a, x) = [(a, b) \mid b \leftarrow x]$.

5.6 $\text{setify} \triangleq (\llbracket \omega, (\tau \times 1) \text{ cup} \rrbracket) : \text{list } \alpha \rightarrow [\alpha]$, a list’s set of elements; ω returns the empty set.

Ex 5.21 $\text{nil setify} = \omega$, $\text{wrap setify} = \tau$, $\text{cat setify} = (\text{setify} \times \text{setify}) \text{ cup}$, $(\text{list } f) \text{ setify} = \text{setify } (Pf)$.

$\text{concat setify} = (\text{list setify}) \text{ setify} \bigcup$, $\text{cpr setify} = (1 \times \text{setify}) \Lambda(1 \times \exists)$.

Ex 5.22 $\text{subseqs setify} = \Lambda \text{ subseq}$ — the list program implements the set specification. By fusion, from the identities above.

Ex 5.23 subseq is the converse of a catamorphism, the one computing supersequences.

Cartesian product

5.6 $\text{cpp}(x, y) = [(a, b) \mid a \leftarrow x, b \leftarrow y]$ and $\text{cpl}(x, b) = [(a, b) \mid a \leftarrow x]$, with $\text{cpp setify} = \Lambda(\exists \times \exists)$ and $\text{cpl setify} = \Lambda(\exists \times 1)$.

5.6 $\text{cp}(F) \triangleq \Lambda(F\exists) : F[\alpha] \rightarrow [F\alpha]$ — one choice from each slot. $\text{cpr}, \text{cpp}, \text{cpl}$ implement it for $FA = \alpha \times A$, $A \times A$ and $A \times \beta$.

$\text{cp}(\text{list}) = \langle \Lambda[\text{nil}, (\exists \times \exists)\text{cons}] \rangle = \langle \text{nil } \tau, \Lambda(\exists \times \exists)(P \text{ cons}) \rangle$ by Eilenberg–Wright;
 $\text{cp}(\text{list})[x_1, \dots, x_n] = \{[a_1, \dots, a_n] \mid a_j \in x_j\}$. On lists: $\text{cplist} = \langle \text{nil wrap}, \text{cpp}(\text{list cons}) \rangle$.

Ex 5.20 $\checkmark$ $\Lambda(R \cup S) = \langle \Lambda R, \Lambda S \rangle \text{ cup}$, $\Lambda(R \cap S) = \langle \Lambda R, \Lambda S \rangle \text{ cap}$, $\Lambda(R \times S) = (\Lambda R \times \Lambda S) \text{ cross}$, with $\text{cup} = \Lambda((\pi \exists) \cup (\pi' \exists))$ and $\text{cap} = \Lambda((\pi \exists) \cap (\pi' \exists))$.

Prefix and suffix

5.6 $\text{prefix} = \text{cat}^\circ \pi$: $x \text{ prefix } y$ when $x \# z = y$ for some z . Also $\text{prefix} = \text{init}^*$ with $\text{init} = \text{snoc}^\circ \pi$, and $\text{prefix} = \langle \text{nil}, \text{nil} \cup \text{cons} \rangle$ — the second nil being $! \text{nil} : \alpha \times \text{list } \alpha \rightarrow \text{list } \alpha$.

$\Lambda \text{ prefix} = \langle \text{nil } \tau, \langle \text{nil } \tau, \Lambda(1 \times \exists)(P \text{ cons}) \rangle \text{ cup} \rangle$.

5.6 $\text{inits} = \langle \text{nil wrap}, \langle \text{nil wrap}, \text{cpr}(\text{list cons}) \rangle \text{ cat} \rangle$, i.e. $\text{inits } [] = [[]]$ and $\text{inits } ([a] \# x) = [[]] \# [[a] \# y \mid y \leftarrow \text{inits } x]$ — initial segments in increasing order of length.

5.6 $\text{suffix} = \text{tail}^*$ with $\text{tail} = \text{cons}^\circ \pi'$, the snoc-list dual of prefix. $\text{tails } [] = [[]]$ and $\text{tails } (x \# [a]) = [y \# [a] \mid y \leftarrow \text{tails } x] \# [[]]$ — decreasing length.

Not implementable as it stands; as a cons-list catamorphism $\text{tails} = \langle \text{nil wrap}, \text{extend} \rangle$ with $\text{extend}(a, [x] \# xs) = [[a] \# x] \# [x] \# xs$.

5.6 $\text{splits} = \langle \text{inits}, \text{tails} \rangle \text{ zip}$ implements $\Lambda(\text{cat}^\circ)$, every way of splitting a list in two — and works only because inits and tails order their segments oppositely.

Ex 5.24 $\text{init} : \text{list}^+ \alpha \rightarrow \text{list } \alpha$ is itself a catamorphism.

Partitions

5.6 $\text{partition} \triangleq \text{concat}^\circ : \text{list } \alpha \rightarrow \text{list}(\text{list}^+ \alpha)$ — cut a list into non-empty contiguous segments. $\text{Rel } \text{partition } [1, 2, 3] = \{[[1], [2], [3]], [[1, 2], [3]], [[1], [2, 3]], [[1, 2, 3]]\}$.

As a catamorphism $\text{partition} = \langle \text{nil}, \text{new} \cup \text{glue} \rangle$ with $\text{new} = (\text{wrap} \times 1)\text{cons}$ and $\text{glue} = (1 \times \text{cons}^\circ) \text{assocl}(\text{cons} \times 1) \text{cons}$: at each step either open a segment or glue the element onto the first one. Pointwise $\text{new}(a, xs) = [[a]] \# xs$ and $\text{glue}(a, [x] \# xs) = [[a] \# x] \# xs$.

- 6 The two agree by the converse-catamorphism theorem of the next chapter: $\text{ran } R = 1$ and $Rf \subset F(1, f)$ in $\implies f^\circ = \langle R \rangle$, applied to $f = \text{concat}$ and $R = [\text{nil}, \text{new} \cup \text{glue}]$.

Ex 5.25 Its surjectivity rests on $\text{new}^\circ \text{new} = \text{cons}^\circ(\text{wrap}^\circ \text{wrap} \times 1) \text{cons}$ and $\text{glue}^\circ \text{glue} = \text{cons}^\circ(\text{cons}^\circ \text{cons} \times 1) \text{cons}$.

Λ partition $= \langle \Lambda \text{ nil}, \Lambda((1 \times \exists)(\text{new} \cup \text{glue})) \rangle$, whose step simplifies (Ex 5.27) to $\Lambda(1 \times \exists) P(\langle \text{new } \tau, \Lambda \text{ glue} \rangle \text{cup}) \bigcup$.

- 5.6 partitions $= \langle \text{nil wrap}, \text{cpr}(\text{list}(\langle \text{new}, \text{glues} \rangle \text{cons})) \text{concat} \rangle$, where glues implements $\Lambda \text{ glue}$: $\text{glues}(a, []) = []$ and $\text{glues}(a, [x] \# xs) = [[a] \# x] \# xs$.

Ex 5.26 partitions setify $= \Lambda$ partition.

Permutations

- 5.6 bag α is the initial algebra $[\text{bnil}, \text{bcons}]$ of $F(A, B) = 1 + A \times B$ whose bcons commutes, $(1 \times \text{bcons})\text{bcons} = \text{exch}(1 \times \text{bcons}) \text{bcons}$, i.e. $\text{bcons}(a, \text{bcons}(b, x)) = \text{bcons}(b, \text{bcons}(a, x))$: order is irrelevant, duplicates are not. $\text{exch} = \text{assocl}(\text{swap} \times 1) \text{assocr}$.

- 5.6 $\text{bagify} = \langle \text{bnil}, \text{bcons} \rangle : \text{list } \alpha \rightarrow \text{bag } \alpha$ is surjective, $\text{bagify}^\circ \text{bagify} = 1$.

- 5.6 $\text{perm} \triangleq \text{bagify} \text{bagify}^\circ$ — equal bags of values. At once $\text{perm} = \text{perm}^\circ$.

As a catamorphism $\text{perm} = \langle \text{nil}, \text{add} \rangle$ with $\text{add} = \text{cons perm}$, or with the usable $\text{add} = (1 \times \text{cat}^\circ) \text{exch}(1 \times \text{cons}) \text{cat}$: $\text{add}(a, x) = y \# [a] \# z$ where $y \# z = x$, so a goes in anywhere.

Ex 5.28 $\text{cons perm} = (1 \times \text{perm}) \text{cons perm}$, from $\text{bagify}^\circ \text{bagify} = 1$.

- 5.6 $\text{adds}(a, x) = [y \# [a] \# z \mid (y, z) \leftarrow \text{splits } x]$ and $\text{perms} = \langle \text{nil wrap}, \text{cpr}(\text{list add}) \rangle$.

Ex 5.29 interleave , all interleavings of two lists, is the converse of a catamorphism.

5.7 Lax natural transformations

Some equalities of the functional world become inequalities in the relational one.

(5.13) ✓ **LAX NATURAL** $\varphi : F \hookrightarrow G \triangleq \varphi(FR) \supset (GR)\varphi$ for every R , where $\varphi_\alpha : G\alpha \rightarrow F\alpha$. The $\supset$ is the way the hook points.

$$\begin{array}{ccc}
 G\alpha & \xrightarrow{\varphi} & F\alpha \\
 GR \downarrow & \supset & \downarrow FR \\
 G\beta & \xrightarrow{\varphi} & F\beta
 \end{array}$$

5.7 Example: $\ni : 1 \hookrightarrow P$, that is $(PR)\ni \subset \ni R$ — immediate from the definition of PR .

Thm 5.2 ✓ F, G relators: φ is lax natural $F \hookrightarrow G \iff \varphi$ is strictly natural on maps, $\varphi(Ff) = (Gf)\varphi$. So lax naturality adds nothing to naturality of the underlying map functors — it only records how it extends.

Ex 5.30 Three instances: $R\tau \subset \tau(PR)$, $R\langle 1, 1 \rangle \subset \langle 1, 1 \rangle (R \times R)$, $(PR \times PR) \text{ cup} \subset \text{cup} (PR)$.